UNIVERSIDADE DO ALGARVE
FACULDADE DE CIÊNCIAS E TECNOLOGIA

Algoritmos e Estruturas de Dados

A • Exame Época Normal • 13 de janeiro de 2025 • 15:00 – 17:00

- Para perguntas com resposta de escolha múltipla, respostas erradas com cotação c e n respostas possíveis descontam $-c / (n - 1)$.
- As respostas às perguntas de desenvolvimento deverão ser as mais detalhadas possíveis, indicando os vários passos intermédios da resolução.
- Na mesa em que está a fazer o exame deve ter apenas lápis/caneta, identificação, este enunciado, e folha de teste.

Grupo I: Perguntas de escolha múltipla

1. (0.5) Considere o seguinte código

```
public static Integer identity(Integer i){return i;}

public static void main(String[] args)
{
    Integer aux = new Integer(-1);
    Integer aux2;

    for(int i = 0; i < 5; i++)
    {
        aux2 = identity(aux);
        aux = new Integer(i);
    }

    System.gc();
…
}
```

Quando a linha System.gc() é executada, o coletor de lixo é invocado pela máquina virtual Java. Na execução do código apresentado, quantos objetos foram criados no heap, e quantos foram marcados para remoção?

a) 5 criados, 4 marcados para remoção.
b) 6 criados, 5 marcados para remoção.
c) 11 criados, 9 marcados para remoção.
d) 6 criados, 4 marcados para remoção.
e) 11 criados, 10 marcados para remoção.

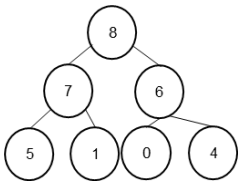
2. (0.5) Quais das afirmações seguintes melhor descreve o funcionamento do algoritmo de ordenação selection sort?

a) Escolhe o maior elemento e coloca-o na posição inicial do array em cada iteração.
b) Divide o array em duas metades, ordena cada metade separadamente, e junta-os ordenadamente.
c) Troca elementos adjacentes fora de ordem até que o array fique ordenado.
d) Em cada iteração, percorre o array, encontra o menor elemento, e coloca-o na posição mais à esquerda não ordenada do array.
e) Efetua recursivamente partições para ordenar parcialmente o array relativamente a um pivot.

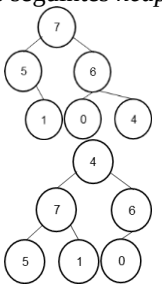
3. (0.5) Considere o algoritmo de ordenação *Bubblesort* aplicado ao seguinte *array* [4, 2, 5, 0, 1, 7, 3, 8]. Indique qual das seguintes sequências corresponde à ordenação do *array* no fim de cada iteração principal do algoritmo.

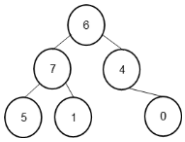
a) [2,4,0,1,5,3,7,8][2,0,1,4,3,5,7,8][0,1,2,3,4,5,7,8][0,1,2,3,4,5,7,8]
b) [2,0,1,4,3,5,7,8][0,1,2,3,4,5,7,8][0,1,2,3,4,5,7,8]
c) [2,4,0,1,3,5,7,8][0,1,2,3,4,5,7,8]
d) [2,4,0,1,3,5,7,8][0,1,2,3,4,5,7,8][0,1,2,3,4,5,7,8]
e) [2,0,1,4,3,5,7,8][0,1,2,4,3,5,7,8][0,1,2,3,4,5,7,8]

4. (0.5) Considere o seguinte *max-heap*:

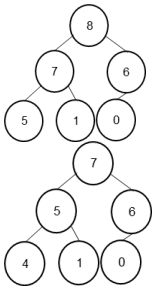


Qual dos seguintes *heaps* corresponde ao resultado de se remover a raiz do heap?

a)


b)


c)

d)


e)

5. (0.5) Quando comparamos tabelas de dispersão com encadeamento separado e tabelas de dispersão com endereçamento aberto, quais das seguintes desvantagens se aplica às tabelas de dispersão com encadeamento separado?

a) Maior complexidade do código devido à utilização de dupla dispersão.
b) Uso adicional de memória.
c) Maior probabilidade de colisões devido à utilização de listas ligadas.
d) Desempenho mau quando a tabela está muito cheia.
e) Incapacidade de reutilizar facilmente espaço de chaves apagadas na tabela.

6. (0.5) Qual o número máximo de arestas em um grafo não dirigido com *n* vértices?

a) n
b) n(n-1)/2
c) n(n-1)
d) 2^n
e) n^2

Grupo II: Perguntas de desenvolvimento

7. (2.5) Exercícios sobre a linguagem de programação Java.

a) (1.0) Escreva um método estático chamado *duplica_impares*, que recebe um *array* de inteiros *a*, e que devolve um novo *array*, em que os elementos ímpares do *array* original foram multiplicados por 2, e os elementos não ímpares foram mantidos. Por exemplo, ao receber o *array* [1,1,2,3,4,5], o método irá retornar o *array* [2,2,2,6,4,10].

b) (1.5) Nesta alínea iremos generalizar o método implementado na alínea anterior. Escreva um método estático de ordem superior, designado *aplica_se*, que recebe um array de inteiros *a*, um operador unário do tipo inteiro (*UnaryOperator<Integer>*) *op*, e um predicado do tipo inteiro (*Predicate<Integer>*) *p*. Este método devolve um novo *array* a partir do *array* original em que os elementos que satisfazem o predicado são transformados usando o operador *op*, e os elementos que não satisfazem o predicado são mantidos no seu valor original no novo *array*. Por exemplo, ao executar a instrução `aplica_se([7,3,2,0,8,9], (i)->i-5, (i)->i>=5)` é retornado o *array* [2,3,2,0,3,4].

Dica: para executar a interface funcional *UnaryOperator* usa-se o método *apply*, e para executar a interface funcional *Predicate* usa-se o método *test*.

8. (1.5) Calcule uma aproximação tilde para a ordem de crescimento (em função de *n*) para a instrução sum++ do seguinte excerto de código. Lembre-se que a soma dos *n* primeiros termos de uma série geométrica é dada por $s_n = \frac{a_1(1-r^n)}{1-r}$.

```
int sum = 0;
for(int i = n; i > 0; i/= 3)
{
    for(int j = 0; j < i; j+=3)
    {
        sum++;
    }
}
```

9. (1.0) Considere a seguinte pilha, em que o topo da pilha contém o número 6.

6
2
8
9
0

Desenhe o estado da pilha resultante das seguintes operações:

pop(), pop(), x=pop(), push(3), push(5), push(7), push(x)

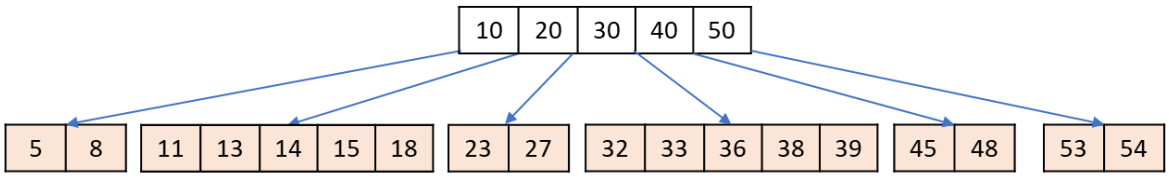
10. (1.5) Considere a seguinte implementação clássica do método partition do algoritmo de ordenação quicksort, onde o pivot v é escolhido como sendo o elemento mais à esquerda do subarray.

```
private static <T extends Comparable<T>> int partition(T[] a, int low, int high)
{
    T v = a[low];
    int i = low, j = high+1;

    while(true)
    {
        while(less(a[++i],v)) if(i == high) break;
        while(less(v,a[--j])) if(j == low) break;
        if(i >= j) break;
        exchange(a, i, j);
    }
    exchange(a, low, j);
    return j;
}
```

Escreva em Java o código de uma versão equivalente e igualmente eficiente do método partition, mas em que o pivot é escolhido como sendo o elemento mais à direita do subarray. O seu código **não pode aumentar o número de exchanges** efetuados. Se preferir, escreva apenas as linhas de código alteradas.

11. (2.5) Considere a seguinte *B-tree* (designada por *FatTree* no Projeto P3), com grau mínimo g=3.



a) (1.0) Considere a execução da operação *get(25)* sobre a B-tree especificada. Indique a ordem pela qual as chaves da árvore são comparadas com a chave 25 durante a execução.
Importante, existem 3 ordens possíveis para comparar as chaves dentro de um nó: da esquerda para a direita, da direita para a esquerda, e por ordem de uma pesquisa binária. Ao responder a esta alínea, **indique qual a**

ordem utilizada no seu projeto, e responda de acordo com a ordem indicada. Caso não se lembre de qual a ordem utilizada no seu projeto, **responda** à questão utilizando a ordem **da esquerda para a direita**.

b) (1.5) Considere a execução da operação *put(35,x)* sobre a *B-tree* especificada. Desenhe a árvore resultante do processo de inserção, redesenhando a árvore sempre que houver um processo de divisão (*split*) de um nó. Marque com um ***v*** os nós da árvore que foram visitados durante o processo de inserção.

12. (2.5) Considere uma árvore rubro-negra (red-black) inicialmente vazia. Insira a seguinte sequência de chaves na árvore: 10, 9, 8, 7, 6. Desenhe os principais passos intermédios, e indique explicitamente quando efetuar uma das seguintes operações: Rotate Left, Rotate Right, e Flip. Desenhe ligações vermelhas a tracejado e ligações negras com linhas contínuas. Desenhe também a árvore final.

13. (2.5) Considere uma tabela de dispersão de dimensão m = 9, com resolução de colisões por dupla dispersão, e funções de dispersão $h_1(k) = k \bmod 9$, e $h_2(k) = 1 + k \bmod 7$

a) (2.0) Sabendo que inicialmente a tabela se encontra vazia, indique quais os elementos presentes em cada posição do *array* de chaves e de valores, apos a inserção da sequência de pares <chave , valor> abaixo. Indique uma posição vazia com **—**. Apresente os valores calculados para $h_1(k)$ e $h_2(k)$ para cada uma das chaves.

<4 , “a”>, <15 , “b”>, <19 , “c”>, <1 , “d”>, <4 , “e”> , <5 , “f”> , <6, “g”>

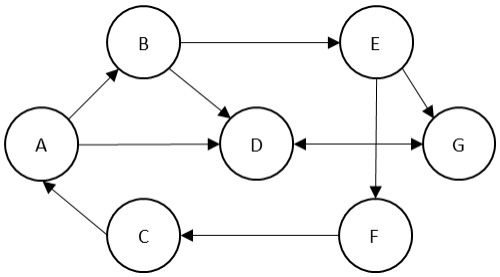
i	0	1	2	3	4	5	6	7	8
chaves									
valores									

b) (1.0) Considere agora que o conteúdo da tabela de dispersão é a seguinte:

i	0	1	2	3	4	5	6	7	8
chaves	—	28	29	—	—	2	—	—	5
valores	—	“a”	“b”	—	—	“c”	—	—	“d”

Indique o que muda na tabela de dispersão, após a remoção da chave *2*.

14. (3.0) Considere o grafo dirigido da figura seguinte:



Aplique o algoritmo de Kosaraju-Sharir para determinar os componentes fortemente ligados do grafo. Desenhe o grafo invertido. Determine a pré-ordem e a pós-ordem invertida do grafo invertido. Finalmente faça uma procura em profundidade primeiro para marcar o componente a que cada vértice pertence. Indique o número de componentes fortemente ligados do grafo, e quais os vértices que correspondem a cada componente. Durante as procuras em profundidade primeiro, quando não existir outro critério para seleção, utilize a ordem alfabética crescente dos nós como critério de seleção. Desenhe os vários passos principais do algoritmo.